

▼ Комп’ютерний практикум 2

Тема: Класичні методи пошуку рішень у просторах станів.

Мета: Розглянути та дослідити алгоритми неінформативного та інформативного пошуку.  
Провести порівняльний аналіз ефективності використання алгоритмів.

**Виконала:** Лемаєва Олександра  
**Група:** IC-32  
**Дата:** 8.11.2025

**Варіант 38 (8)**  
**Task:** Оптимальне розподілення задач на 2 процесори  
**AlgNoInf:** BFS у просторі призначень  
**AlgInf:** A\*  
**Func:** Нижня межа максимального завантаження (makespan)

1. Постановка задачі

У цій роботі я реалізувала задачу оптимального розподілення задач між двома процесорами.  
Є набір задач з різною тривалістю виконання. Метою є - призначити кожну задачу одному з процесорів так, щоб обидва були завантажені максимально рівномірно, тобто мінімізувати максимальне навантаження.  
Початковий стан: жодна задача не призначена (i=0, L1=0, L2=0).  
Цільовий стан: усі задачі розподілені (i=n).  
Оператори: додати поточну задачу або до процесора 1, або до процесора 2.  
Простір станів: усі можливі варіанти розподілення, тобто дерево глибини n (2^n варіантів).

▼ 2. Евристична функція

Для алгоритму A\* використовується **допустима евристика** (нижня межа максимального завантаження (makespan)).  
Вона визначається як:  $LB = \max \left( \max(L_1, L_2), \left\lceil \frac{L_1 + L_2 + R}{2} \right\rceil, \min(L_1, L_2) + t_{\max} \right)$   
де  $R = \sum$  тривалостей ще не призначених задач,  $t_{\max}$  — найбільша з них.  
Ця оцінка ніколи не переоцінює реальний makespan, тому A\* гарантує оптимальне рішення.

▼ 1) Імпорти та типи

```
import math, random, time, heapq
from dataclasses import dataclass
from typing import List, Tuple, Optional, Dict
import statistics as stats
```

▼ 2) Модель стану та нижня межа makespan

Два класи - **State** для поточного стану пошуку та **SearchResult** для результатів (скільки було станів, час, чи завершено).  
Функція lower\_bound повертає оцінку того, який мінімальний makespan можливий із поточного стану.  
Бере поточні навантаження двох процесорів і дивиться на ще не розподілені задачі.

```
@dataclass
class State:
    i: int
    L1: int
    L2: int

    @dataclass
    class SearchResult:
        best_makespan: Optional[int]
        best_assign: Optional[List[int]]
        visited: int
        max_frontier: int
        elapsed_sec: float
        terminated_by_limit: bool

def lower_bound(L1: int, L2: int, rem: List[int]) -> int:
    if not rem:
        return max(L1, L2)
    R = sum(rem)
    tmax = max(rem)
    lb1 = max(L1, L2)
    lb2 = math.ceil((L1 + L2 + R) / 2)
    lb3 = min(L1, L2) + tmax
    return max(lb1, lb2, lb3)
```

▼ 3) BFS (неінформативний пошук)

Алгоритм досліджує простір станів пошарово. Реалізовано найпростіший (без перевірок повторів, відсічень чи покращень).  
Просто перебирає всі варіанти, поки не знайде мінімальний makespan.

```
from collections import deque

def bfs_two_proc(durs: List[int],
                 time_limit_sec: float = 600.0,
                 frontier_cap: int = 30_000_000) -> SearchResult:
    t0 = time.perf_counter()
    n = len(durs)
    q = deque()
    q.append( (State(0,0,0), []) )
    visited = 0
    max_frontier = 1
    best_C = None
    best_assign = None

    while q:
        # nimiri
        if (time.perf_counter() - t0) > time_limit_sec or len(q) > frontier_cap:
            return SearchResult(best_C, best_assign, visited, max_frontier,
                                time.perf_counter() - t0, True)

        st, assign = q.popleft()
        visited += 1

        if st.i == n:
            C = max(st.L1, st.L2)
            if best_C is None or C < best_C:
```

```

        best_C, best_assign = C, assign
        continue

    ti = durs[st.i]
    q.append( (State(st.i+1, st.L1+ti, st.L2), assign + [0]) )
    q.append( (State(st.i+1, st.L1, st.L2+ti), assign + [1]) )
    if len(q) > max_frontier:
        max_frontier = len(q)

return SearchResult(best_C, best_assign, visited, max_frontier,
                    time.perf_counter() - t0, False)

```

#### 4) A\* (інформативний пошук)

Використовую евристику щоб алгоритм не розглядав гілки, які точно не дадуть кращого результату.

Функція `f_score` обчислює нижню межу, якщо вона перевищує найкраще відоме рішення (UB), то гілка відсікається.

```

def a_star_two_proc(durs: List[int],
                    time_limit_sec: float = 600.0,
                    frontier_cap: int = 30_000_000,
                    use_ub: bool = True) -> SearchResult:
    t0 = time.perf_counter()
    n = len(durs)

    suffix = [0]*(n+1)
    for i in range(n-1,-1,-1):
        suffix[i] = suffix[i+1] + durs[i]
    suf_max = [0]*(n+1)
    for i in range(n-1,-1,-1):
        suf_max[i] = max(suf_max[i+1], durs[i])

    def f_score(i,L1,L2):
        rem_sum = suffix[i]
        rem_max = suf_max[i]
        lb = max( max(L1,L2),
                  math.ceil((L1+L2+rem_sum)/2),
                  min(L1,L2)+rem_max )
        return lb

    UB = None
    if use_ub:
        loads = [0,0]
        for t in sorted(durs, reverse=True):
            j = 0 if loads[0] <= loads[1] else 1
            loads[j] += t
        UB = max(loads)

    heap = []
    heapq.heappush(heap, (f_score(0,0,0), 0, 0, 0, []))
    visited = 0
    max_frontier = 1
    best_C = None
    best_assign = None

    while heap:
        if (time.perf_counter() - t0) > time_limit_sec or len(heap) > frontier_cap:
            return SearchResult(best_C, best_assign, visited, max_frontier,
                                time.perf_counter() - t0, True)

        f,i,L1,L2,assign = heapq.heappop(heap)
        visited += 1

        if i == n:
            C = max(L1,L2)
            if best_C is None or C < best_C:
                best_C, best_assign = C, assign
            if UB is None or C < UB:
                UB = C
            continue

        if UB is not None and f > UB:
            continue

        ti = durs[i]
        for put_on_p2 in (0,1):
            nL1 = L1 + (ti if put_on_p2==0 else 0)
            nL2 = L2 + (ti if put_on_p2==1 else 0)
            new_assign = assign + [put_on_p2]
            nf = f_score(i+1, nL1, nL2)
            if UB is None or nf <= UB:
                heapq.heappush(heap, (nf, i+1, nL1, nL2, new_assign))
            if len(heap) > max_frontier:
                max_frontier = len(heap)

    return SearchResult(best_C, best_assign, visited, max_frontier,
                        time.perf_counter() - t0, False)

```

#### 5) Серія експериментів і підрахунок середніх значень

Створюю кілька випадкових наборів задач і запускаю на них обидва алгоритми BFS та A\*. Після кожного запуску зберігаються всі потрібні метрики: час, кількість відвіданих станів та максимальний розмір черги.

Функція `pretty_table` виводить результати у зручному табличному форматі, а `plot_series` будувє графік, який показує як відрізняється масштаб пошуку між BFS і A\*.

```

def gen_instance(n: int, lo:int=1, hi:int=20, rnd:random.Random=None) -> List[int]:
    rnd = rnd or random
    return [rnd.randint(lo,hi) for _ in range(n)]

def run_series(alg_name: str, runner, instances: List[List[int]], **opts):
    rows = []
    for k, durs in enumerate(instances, 1):
        res = runner(durs, **opts)
        rows.append({
            "exp": k,
            "n": len(durs),
            "sum": sum(durs),
            "time": res.elapsed_sec,
            "visited": res.visited,
            "max_frontier": res.max_frontier,
            "bestC": res.best_makespan,
            "cut": int(res.terminated_by_limit)
        })
    avg_time = stats.fmean(r["time"] for r in rows)
    avg_vis = stats.fmean(r["visited"] for r in rows)
    avg_mem = stats.fmean(r["max_frontier"] for r in rows)
    solved = sum(1 for r in rows if r["bestC"] is not None)
    return rows, {"alg":alg_name, "avg_time":avg_time, "avg_visited":avg_vis,
                  "avg_frontier":avg_mem, "solved":solved, "total":len(rows)}

def pretty_table(rows, title):
    print(f"\n{title}")
    print("exp | n | sum | time(s) | visited | max_frontier | bestC | cut")
    print("-"*65)
    for r in rows:

```

```
print(f"{r['exp']:>3} | {r['n']:>2} | {r['sum']:>3} | "
      f"{r['time']:.4f} | {r['visited']:>7} | {r['max_frontier']:>12} | "
      f"{str(r['bestC']):>5} | {r['cut']]}")
print("--"65)

def plot_series(bfs_rows, a_rows):
    import matplotlib.pyplot as plt
    xs = [r["exp"] for r in bfs_rows]
    plt.figure()
    plt.plot(xs, [r["visited"] for r in bfs_rows], label="BFS visited")
    plt.plot(xs, [r["visited"] for r in a_rows], label="A* visited")
    plt.xlabel("Експеримент")
    plt.ylabel("Кількість згенерованих/вибраних станів")
    plt.title("BFS vs A* – розмір пошуку")
    plt.legend()
    plt.tight_layout()
    plt.show()
```

6) Підсумковий запуск серії експериментів

Фіксуємо seed, генеруємо не менше 20 випадкових інстансів (різний розмір і тривалості) і запускаємо обидва алгоритми на кожному. Функція збирає метрики по всіх експериментах, друкує таблиці та середні показники. Наприкінці викликаємо plot\_series, щоб показати різницю між BFS та A\* на графіку. Останній рядок run\_all() просто стартує всю серію з типовими параметрами.

```
def run_all(seed=42, experiments=20, n_min=12, n_max=18,
            dur_lo=1, dur_hi=20, bfs_time_limit=30.0, a_time_limit=30.0):
    rnd = random.Random(seed)
    instances = [gen_instance(rnd.randint(n_min,n_max), dur_lo, dur_hi, rnd) for _ in range(experiments)]

    bfs_rows, bfs_sum = run_series("BFS", bfs_two_proc, instances,
                                   time_limit_sec=bfs_time_limit)
    a_rows, a_sum      = run_series("A*", a_star_two_proc, instances,
                                   time_limit_sec=a_time_limit)

    pretty_table(bfs_rows, "BFS: результати")
    pretty_table(a_rows, "A*: результати")

    print("\n Середні показники")
    print(f"BFS: time={bfs_sum['avg_time']:.4f}s, visited={bfs_sum['avg_visited']:.1f}, "
          f"maxFrontier={bfs_sum['avg_frontier']:.1f}, solved={bfs_sum['solved']}/{bfs_sum['total']}")
    print(f"A*:  time={a_sum['avg_time']:.4f}s, visited={a_sum['avg_visited']:.1f}, "
          f"maxFrontier={a_sum['avg_frontier']:.1f}, solved={a_sum['solved']}/{a_sum['total']}")

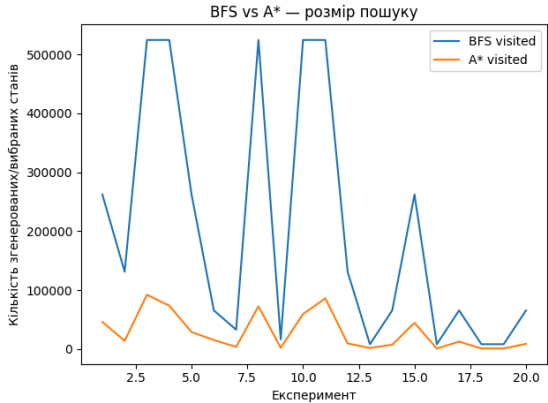
    try:
        plot_series(bfs_rows, a_rows)
    except Exception:
        print("(matplotlib недоступний)")

run_all()
```

| BFS: результати |    |     |         |         |              |       |     |
|-----------------|----|-----|---------|---------|--------------|-------|-----|
| exp             | n  | sum | time(s) | visited | max_frontier | bestC | cut |
| 1               | 17 | 131 | 0.7217  | 262143  | 131072       | 66    | 0   |
| 2               | 16 | 162 | 0.3067  | 131071  | 65536        | 81    | 0   |
| 3               | 18 | 191 | 1.5451  | 524287  | 262144       | 96    | 0   |
| 4               | 18 | 162 | 1.5020  | 524287  | 262144       | 81    | 0   |
| 5               | 17 | 167 | 0.9331  | 262143  | 131072       | 84    | 0   |
| 6               | 15 | 163 | 0.2082  | 65535   | 32768        | 82    | 0   |
| 7               | 14 | 143 | 0.1413  | 32767   | 16384        | 72    | 0   |
| 8               | 18 | 213 | 1.9735  | 524287  | 262144       | 107   | 0   |
| 9               | 13 | 140 | 0.0203  | 16383   | 8192         | 70    | 0   |
| 10              | 18 | 175 | 1.6073  | 524287  | 262144       | 88    | 0   |
| 11              | 18 | 213 | 1.5576  | 524287  | 262144       | 107   | 0   |
| 12              | 16 | 136 | 0.2385  | 131071  | 65536        | 68    | 0   |
| 13              | 12 | 134 | 0.0590  | 8191    | 4096         | 67    | 0   |
| 14              | 15 | 143 | 0.1457  | 65535   | 32768        | 72    | 0   |
| 15              | 17 | 167 | 0.6854  | 262143  | 131072       | 84    | 0   |
| 16              | 12 | 110 | 0.0092  | 8191    | 4096         | 55    | 0   |
| 17              | 15 | 164 | 0.1400  | 65535   | 32768        | 82    | 0   |
| 18              | 12 | 118 | 0.0094  | 8191    | 4096         | 59    | 0   |
| 19              | 12 | 139 | 0.0091  | 8191    | 4096         | 70    | 0   |
| 20              | 15 | 167 | 0.1365  | 65535   | 32768        | 84    | 0   |

| A*: результати |    |     |         |         |              |       |     |
|----------------|----|-----|---------|---------|--------------|-------|-----|
| exp            | n  | sum | time(s) | visited | max_frontier | bestC | cut |
| 1              | 17 | 131 | 0.1720  | 45979   | 12802        | 66    | 0   |
| 2              | 16 | 162 | 0.1012  | 13945   | 3411         | 81    | 0   |
| 3              | 18 | 191 | 0.3066  | 92029   | 27085        | 96    | 0   |
| 4              | 18 | 162 | 0.3834  | 73689   | 21788        | 81    | 0   |
| 5              | 17 | 167 | 0.0971  | 28755   | 6338         | 84    | 0   |
| 6              | 15 | 163 | 0.0499  | 15377   | 5604         | 82    | 0   |
| 7              | 14 | 143 | 0.0104  | 3593    | 926          | 72    | 0   |
| 8              | 18 | 213 | 0.3144  | 72505   | 22194        | 107   | 0   |
| 9              | 13 | 140 | 0.0057  | 2077    | 452          | 70    | 0   |
| 10             | 18 | 175 | 0.2253  | 59255   | 11058        | 88    | 0   |
| 11             | 18 | 213 | 0.4075  | 86065   | 26368        | 107   | 0   |
| 12             | 16 | 136 | 0.0283  | 9513    | 1704         | 68    | 0   |
| 13             | 12 | 134 | 0.0049  | 1777    | 736          | 67    | 0   |
| 14             | 15 | 143 | 0.0217  | 7309    | 1224         | 72    | 0   |
| 15             | 17 | 167 | 0.1815  | 44409   | 11504        | 84    | 0   |
| 16             | 12 | 110 | 0.0023  | 861     | 188          | 55    | 0   |
| 17             | 15 | 164 | 0.0587  | 12571   | 5326         | 82    | 0   |
| 18             | 12 | 118 | 0.0027  | 969     | 240          | 59    | 0   |
| 19             | 12 | 139 | 0.0025  | 991     | 196          | 70    | 0   |
| 20             | 15 | 167 | 0.0264  | 8751    | 2138         | 84    | 0   |

Середні показники  
BFS: time=0.5975s, visited=200703.0, maxFrontier=100352.0, solved=20/20  
A\*: time=0.1238s, visited=29021.0, maxFrontier=8064.1, solved=20/20



#### 4. Аналіз результатів

Завдяки нижній межі makespan алгоритм A\* значно ефективніший і показує реальну користь інформативного пошуку. Після проведення серії з 20 експериментів я порівняла роботу алгоритмів. Обидва успішно знаходять оптимальний makespan у всіх випадках (тобто вони повні та гарантують правильний результат). Проте відмінності у швидкості та обсязі пошуку виявилися дуже суттєвими.

Під час виконання експериментів BFS показав, що йому доводиться перебирати майже весь простір станів. BFS дуже сильно залежить від розміру простору і практично не має ніяких механізмів, які б дозволяли уникати неперспективних гілок.

На відміну від нього алгоритм A\* працює значно ефективніше. Завдяки використанню евристики нижньої межі makespan він одразу відсікає багато гілок, які не можуть привести до кращого розв'язку.

Обидва алгоритми завжди знаходять оптимальний розподіл задач між процесорами, але роблять це по-різному. A\* працює набагато обережніше та більш вибірково. Можемо це побачити на графіку - лінія BFS різко стрибає до великих значень, а A\* тримається значно нижче, що підтверджує ефективність інформативного пошуку.

Отже, проведені експерименти показали, що A\* є більш придатним алгоритмом оскільки дає ті самі оптимальні результати, але при цьому значно економить час та пам'ять.